EXPLORING AUTHORIZATION BEFORE RETRIEVAL IN PRACTICE

The architectural pattern described in this section is implemented concretely in a companion GitHub repository, `cedar-rag-authz-demo` (https://github.com/windley/cedar-rag-authz-demo/tree/main). The repository includes a Jupyter notebook that walks through a simplified RAG pipeline and demonstrates how authorization-before-retrieval works in practice.

The notebook focuses on a single technical idea: partial authorization requests can be transformed into retrieval constraints, rather than simple `allow` or `deny` results.

Cedar typed partial evaluation

The demonstration uses Cedar's typed partial evaluation (TPE) to make this possible. In a normal authorization request, a policy engine evaluates a policy with a complete set of inputs and returns a decision. With TPE, some inputs are purposely left unspecified. Instead of delivering a final decision, the policy engine generates a residual, a partially evaluated expression that shows the remaining constraints under which access would be allowed.

In the context of RAG, those remaining constraints define the authorized scope of data rather than a single access decision. For example, a residual might specify conditions such as "documents owned by customers in this workspace" or "records visible to managers of this account." These conditions are precise, deterministic, and based on policy, even if they are not yet linked to specific resources.

From policy residuals to retrieval filters

The notebook shows how these policy residuals can be turned into specific retrieval filters for the Amazon OpenSearch vector database. Instead of broadly querying a vector database and letting the language model filter results, the system uses the residual as part of the retrieval query. Only records within the authorized scope are retrieved.